Bài 2.12 – SVM (Support Vector Machine)

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *Đố vui trước khi học*

🎯 Sau bài này bạn sẽ

- Hiểu ý tưởng **lề rộng nhất (maximum margin)** và **support vector**.
- Tự kiểm tra được đường biên và độ rộng lề trên ví dụ nhỏ.
- Hiểu **kernel trick**: cách SVM vẽ được đường biên cong mà không cần tạo đặc trưng bằng tay.
- Điều khiển hai tham số quan trọng **C** và **gamma**, và nhận ra overfitting qua hình dạng đường biên.
- Biết khi nào SVM là lựa chọn tốt, khi nào không.

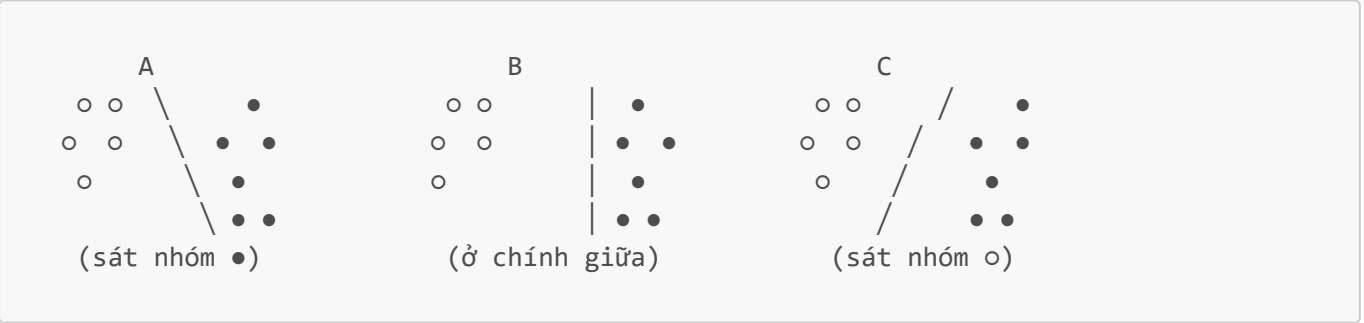
Chia buổi:

- **Buổi 1:** Mục 1 → 4 (lề, support vector, C).
- **Buổi 2:** Mục 5 → 9 (kernel, gamma, dữ liệu thật, bài tập).

---

1. Đố vui trước khi học 🤔

Có hai nhóm điểm trên mặt phẳng: ○ và ●. Ba người vẽ ba đường thẳng, **cả ba đều tách đúng 100%** dữ liệu train:



Câu hỏi: Có một điểm **mới** sắp xuất hiện. Bạn tin đường nào nhất? Vì sao?

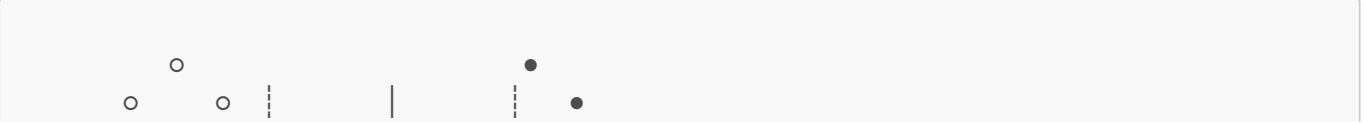
Hầu hết mọi người chọn **B**: nó cách **đều và xa nhất** cả hai nhóm, nên nếu điểm mới hơi lệch một chút, vẫn ít khả năng bị phân loại sai. Trực giác này chính là toàn bộ ý tưởng của SVM.

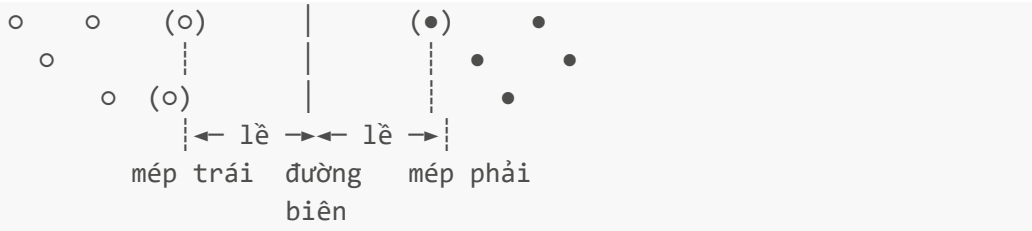
---

2. Ý tưởng: con đường rộng nhất

2.1. Khái niệm

- **SVM – Support Vector Machine (Máy vector hỗ trợ):** Tìm đường biên tách hai lớp sao cho **khoảng trống hai bên đường (lề) rộng nhất**.
- **Ví dụ đời thực:** Bạn cần làm một **con đường** chạy giữa hai ngôi làng. Muốn đường càng **rộng** càng tốt mà không đè lên nhà nào. Tim đường là đường biên. Mép đường chạm vào vài ngôi nhà gần nhất.





(○) và (●): các SUPPORT VECTOR, những điểm nằm ngay trên mép đường

2.2. Support vector (Vector hỗ trợ)

- **Định nghĩa:** Các điểm train nằm **sát đường biên nhất** (trên mép lề hoặc lấn vào lề).
- **Điều bất ngờ:** Đường biên **chỉ phụ thuộc vào các support vector**. Xóa mọi điểm khác đi, đường biên **không đổi**. Đó là lý do có tên "máy vector hỗ trợ": vài điểm này "chống đỡ" toàn bộ đường biên.

2.3. Kiểm tra trên ví dụ nhỏ

Lớp 0: (1, 1), (2, 1), (1, 2). Lớp 1: (4, 4), (5, 4), (4, 5).

```
import numpy as np
from sklearn.svm import SVC

Xd = np.array([[1, 1], [2, 1], [1, 2], [4, 4], [5, 4], [4, 5]], dtype=float)
yd = np.array([0, 0, 0, 1, 1, 1])
s = SVC(kernel="linear", C=1000).fit(Xd, yd)      # kernel="linear": đường biên
thẳng
print(s.support_vectors_)                          # các support vector
print(s.coef_.round(3), s.intercept_.round(3))     # w và b của đường biên w·x + b
= 0
print("Độ rộng lề:", round(2 / np.linalg.norm(s.coef_[0]), 3))
```

Kết quả mẫu:

```
[[2. 1.]
 [1. 2.]
 [4. 4.]]
[[0.4 0.4]] [-2.2]
Độ rộng lề: 3.536
```

Kiểm tra bằng tay:

- Đường biên: $0{,}4 \cdot x_1 + 0{,}4 \cdot x_2 - 2{,}2 = 0 \Leftrightarrow \mathbf{x_1 + x_2 = 5{,}5}$.
- Support vector lớp 0: (2, 1) và (1, 2) đều có $x_1 + x_2 = \mathbf{3}$. Lớp 1: (4, 4) có $x_1 + x_2 = \mathbf{8}$. Đường biên nằm **chính giữa**: (3 + 8) / 2 = 5,5. ✓
- Khoảng cách giữa hai đường $x_1 + x_2 = 3$ và $x_1 + x_2 = 8$ là $|8 - 3| / \sqrt{1^2 + 1^2} = 5 / 1{,}414 \approx \mathbf{3{,}536}$. ✓ (Công thức tổng quát: độ rộng lề = 2 / ||w||.)
- Điểm (1, 1) và (5, 4), (4, 5) nằm xa hơn, **không ảnh hưởng** gì tới đường biên.

---

3. Lề mềm và tham số C

3.1. Vấn đề

Dữ liệu thật hiếm khi tách được hoàn hảo: luôn có vài điểm "lạc đàn" (nhiễu, outlier). Nếu bắt đường biên tách đúng 100%, nó sẽ uốn theo từng điểm lạc → overfit (bài 2.6).

3.2. Soft margin (Lề mềm)

Cho phép **một số điểm vi phạm lề** (nằm trong lề hoặc bên kia đường), nhưng **phạt** mỗi vi phạm. Tham số **C** quyết định mức phạt:

C	Ý nghĩa	Hậu quả
Nhỏ	"Vi phạm cũng được, miễn đường rộng"	Lề rộng, nhiều support vector, đường biên mượt → có thể **underfit**
Lớn	"Không được phép sai"	Lề hẹp, đường biên uốn theo từng điểm → có thể **overfit**

C trong SVM giống C trong Logistic Regression (bài 2.6): **C = 1/α**, C càng nhỏ, regularization càng mạnh.

---

4. Thí nghiệm với dữ liệu "hai mặt trăng"

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = make_moons(n_samples=400, noise=0.2, random_state=42)   # 2 lớp hình lưỡi liềm lồng vào nhau
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42)

for C in [0.01, 1, 100, 10000]:
    m = SVC(kernel="rbf", C=C).fit(Xtr, ytr)     # kernel "rbf" sẽ học ở mục 5
    print(f"C = {C:>6}: acc train = {m.score(Xtr, ytr):.3f} | test = {m.score(Xte, yte):.3f}"
          f" | số support vector = {m.n_support_.sum()}")
```

Kết quả mẫu:

```
C =   0.01: acc train = 0.839 | test = 0.825 | số support vector = 280
C =      1: acc train = 0.968 | test = 0.950 | số support vector = 66
C =    100: acc train = 0.975 | test = 0.967 | số support vector = 27
C =  10000: acc train = 0.982 | test = 0.958 | số support vector = 31
```

- C = 0,01: lề rất rộng, **280 / 280** điểm train đều là support vector (đều nằm trong lề) → underfit.
- C tăng: lề hẹp lại, số support vector giảm, train tăng.
- C quá lớn (10.000): train tiếp tục tăng, test bắt đầu giảm → dấu hiệu overfit.

5. Kernel trick: đường biên cong

5.1. Vấn đề

Hai mặt trăng lồng vào nhau: **không đường thẳng nào** tách được.

```
for ten, mh in [("Logistic", LogisticRegression()),
                ("SVM linear", SVC(kernel="linear")),
                ("SVM poly 3", SVC(kernel="poly", degree=3)),
                ("SVM RBF", SVC(kernel="rbf"))]:
    mh.fit(Xtr, ytr)
    print(f"{ten:<12}: acc test = {mh.score(Xte, yte):.3f}")
```

Kết quả mẫu:

```
Logistic    : acc test = 0.817
SVM linear  : acc test = 0.825
SVM poly 3  : acc test = 0.867
SVM RBF     : acc test = 0.950
```

Các mô hình đường thẳng kẹt ở khoảng 82%. SVM với kernel RBF đạt **95%**.

5.2. Trực giác: nâng lên chiều cao hơn

Ví dụ 1 chiều: các điểm trên trục số. Lớp ● ở giữa (|x| ≤ 1), lớp ○ ở hai bên.

```
1 chiều: không có điểm cắt nào tách được ● khỏi ○
  ○     ○     ●     ●     ●     ○     ○
──┼─────┼─────┼─────┼─────┼─────┼─────┼──► x
 -3    -2    -1     0     1     2     3
```

Thêm một chiều mới **x^2**: mỗi điểm (x) thành (x, x^2). Với x = −3, −2, −1, 0, 1, 2, 3, ta có x^2 = 9, 4, 1, 0, 1, 4, 9.

```
2 chiều (x, x²): giờ có thể kẻ một đường NGANG tách hai lớp!
x² ▲
 9 ┤ ○                   ○
   │
 4 ┤       ○           ○
 2 ┤- - - - - - - - - - - - - - - - - ← đường thẳng x² = 2
 1 ┤             ●   ●   ← (thật ra (−1,1) và (1,1))
 0 ┤               ●
   └──┼─────┼─────┼─────┼─────┼──► x
     -3    -2     0     2     3
```

Đường thẳng $x^2 = 2$ trong không gian mới, khi "chiếu" về trục x ban đầu, chính là **hai điểm cắt** $x = \pm 1{,}41$: một đường biên "cong" theo nghĩa của không gian gốc.

Đó là ý tưởng ở bài 2.6 (thêm cột đa thức). Nhưng với nhiều đặc trưng, số cột mới bùng nổ (8 cột → 165 cột với bậc 3).

5.3. Kernel trick (Mẹo nhân)

Phép toán SVM cần thực chất chỉ là **tích vô hướng** giữa các cặp điểm (bài 1.8). **Kernel** là một hàm tính **thẳng** tích vô hướng **trong không gian mới**, mà **không cần tạo ra** không gian mới đó. Tiết kiệm khổng lồ, và cho phép dùng cả không gian **vô hạn chiều**.

Kernel	Hình dạng đường biên	Ghi chú
`linear`	Đường thẳng / mặt phẳng	Nhanh; tốt khi nhiều đặc trưng (văn bản)
`poly`	Đường cong đa thức	Tham số `degree`
`rbf` (mặc định)	Cong tùy ý, như các "ngọn đồi" quanh điểm train	Phổ biến nhất

5.4. Kernel RBF và tham số gamma

RBF (Radial Basis Function): đo độ giống nhau giữa hai điểm theo khoảng cách:

$$K(x, x') = e^{(-\gamma \cdot \|x - x'\|^2)}$$

Ký hiệu	Ý nghĩa
$\|x - x'\|^2$	Bình phương khoảng cách Euclid (bài 1.8)
γ (gamma)	Mức "tầm ảnh hưởng" của mỗi điểm. γ lớn → mỗi điểm chỉ ảnh hưởng **rất gần** quanh nó
K	Gần 1 khi hai điểm sát nhau, gần 0 khi xa nhau

Tính thử: hai điểm cách nhau 1 đơn vị ($\|x - x'\|^2 = 1$). Với $\gamma = 0{,}1$: $K = e^{(-0{,}1)} \approx 0{,}905$ (vẫn "giống nhau"). Với $\gamma = 10$: $K = e^{(-10)} \approx 0{,}00005$ (coi như "không liên quan").

```
import matplotlib.pyplot as plt

for g in [0.1, 1, 10, 100]:
    m = SVC(kernel="rbf", gamma=g).fit(Xtr, ytr)
    print(f"gamma = {g:>5}: acc train = {m.score(Xtr, ytr):.3f} | test =
{m.score(Xte, yte):.3f}"
          f" | số support vector = {m.n_support_.sum()}")

# Vẽ đường biên với 3 giá trị gamma
xx, yy = np.meshgrid(np.linspace(-1.5, 2.5, 200), np.linspace(-1, 1.5, 200))   #
lưới điểm phủ mặt phẳng
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, g in zip(axes, [0.1, 1, 100]):
    m = SVC(kernel="rbf", gamma=g).fit(Xtr, ytr)
    Z = m.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)   # dự đoán cho
mọi điểm lưới
    ax.contourf(xx, yy, Z, alpha=0.3)                                # tô màu vùng
theo lớp dự đoán
    ax.scatter(Xtr[:, 0], Xtr[:, 1], c=ytr, s=10)
    ax.set_title(f"gamma = {g}")
plt.show()
```

Kết quả mẫu:

```
gamma =   0.1: acc train = 0.843 | test = 0.833 | số support vector = 116
gamma =     1: acc train = 0.964 | test = 0.950 | số support vector = 63
gamma =    10: acc train = 0.982 | test = 0.967 | số support vector = 92
gamma =   100: acc train = 0.996 | test = 0.958 | số support vector = 238
```

Bạn sẽ thấy:

- **gamma = 0,1:** đường biên gần như một đường cong nhẹ, cắt ngang qua giữa hai mặt trăng → underfit.
- **gamma = 1:** đường biên uốn theo đúng hình chữ S giữa hai mặt trăng. Đẹp.
- **gamma = 100:** vùng màu vỡ thành nhiều "hòn đảo" nhỏ bao quanh từng cụm điểm, có cả đảo bao một điểm lạc → overfit. 238 / 280 điểm thành support vector.

Tóm tắt hai tham số:

```
              gamma nhỏ           gamma lớn
              (đường mượt)        (đường lởm chởm, đảo nhỏ)
  C nhỏ       underfit nặng       ...
  (lề rộng)
  C lớn       ...                 overfit nặng
  (lề hẹp)
  → Cần chọn cặp (C, gamma) cùng nhau bằng cross-validation (bài 2.18)
```

---

6. Dữ liệu thật: chẩn đoán ung thư vú

Bộ **Breast Cancer Wisconsin** (có sẵn trong Scikit-learn): 569 mẫu sinh thiết, 30 chỉ số đo từ ảnh tế bào. Nhãn: ác tính (malignant) hay lành tính (benign).

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import roc_auc_score
```

```
bc = load_breast_cancer(as_frame=True)
print(bc.data.shape, bc.target.value_counts().to_dict(), list(bc.target_names))
Xtr, Xte, ytr, yte = train_test_split(bc.data, bc.target, test_size=0.25,
random_state=42, stratify=bc.target)

m0 = SVC().fit(Xtr, ytr)
print("Không chuẩn hóa:", round(m0.score(Xte, yte), 3))
m1 = make_pipeline(StandardScaler(), SVC()).fit(Xtr, ytr)
print("Có chuẩn hóa:", round(m1.score(Xte, yte), 3))
```

Kết quả mẫu:

```
(569, 30) {1: 357, 0: 212} ['malignant', 'benign']
Không chuẩn hóa: 0.923
Có chuẩn hóa: 0.979
```

(Lưu ý: trong bộ này, nhãn **1 = lành tính**, 0 = ác tính.) SVM dựa trên khoảng cách (kernel RBF), nên giống KNN: **bắt buộc chuẩn hóa**.

Xác suất từ SVM

SVM gốc không cho xác suất, chỉ cho **khoảng cách có dấu** tới đường biên (`decision_function`): dương → lớp 1, âm → lớp 0, càng xa 0 càng chắc chắn.

```
print(m1.decision_function(Xte[:3]).round(2), m1.predict(Xte[:3]))
m2 = make_pipeline(StandardScaler(), SVC(probability=True,
random_state=42)).fit(Xtr, ytr)  # bật ước lượng xác suất
print(round(roc_auc_score(yte, m2.predict_proba(Xte)[:, 1]), 4),
      round(roc_auc_score(yte, m1.decision_function(Xte)), 4))   # AUC chỉ cần thứ
hạng → dùng được decision_function
```

Kết quả mẫu:

```
[ 1.33 -2.3   0.15] [1 0 1]
0.9969 0.9969
```

- Mẫu thứ 3 có giá trị **0,15**: rất gần đường biên → mô hình **không chắc**. Với chẩn đoán y tế, những ca như vậy nên được bác sĩ xem lại.
- `probability=True` làm huấn luyện **chậm hơn** (phải chạy thêm cross-validation bên trong). Nếu chỉ cần AUC hoặc xếp hạng, dùng `decision_function` là đủ (bài 2.5: AUC chỉ phụ thuộc thứ hạng).

So sánh với Logistic Regression trên cùng dữ liệu: `make_pipeline(StandardScaler(), LogisticRegression())` đạt accuracy **0,986**. Với 30 đặc trưng khá "thẳng", mô hình tuyến tính đã rất tốt.

---

7. Ưu, nhược điểm

Ưu điểm	Nhược điểm
Mạnh với dữ liệu **ít mẫu, nhiều đặc trưng**	**Chậm** khi nhiều mẫu (trên ~50.000 dòng gần như không dùng được với kernel RBF)
Kernel cho đường biên cong linh hoạt	Bắt buộc chuẩn hóa
Chỉ phụ thuộc support vector → khá chịu nhiễu ở xa biên	Phải chỉnh cặp C, gamma cẩn thận
Có cơ sở toán học vững	Không cho xác suất trực tiếp; khó giải thích

Vị trí ngày nay: Trước năm 2012, SVM là "ngôi sao" của ML (nhận dạng chữ viết tay, phân loại văn bản). Ngày nay, với dữ liệu bảng lớn, Gradient Boosting thường tốt hơn; với ảnh và văn bản, Deep Learning thắng thế. SVM vẫn rất đáng dùng cho **dữ liệu nhỏ, nhiều chiều**, như dữ liệu y sinh học. Có thêm **SVR** (`sklearn.svm.SVR`) cho hồi quy và `LinearSVC` cho dữ liệu văn bản lớn.

---

8. Bài tập

1. Trong ví dụ nhỏ ở mục 2.3, thêm điểm (3, 3) vào **lớp 0**. Dự đoán (không chạy code): support vector và đường biên sẽ thay đổi thế nào? Sau đó chạy để kiểm tra.
2. **Tính tay** giá trị kernel RBF giữa hai điểm (0, 0) và (1, 2) với $\gamma = 0{,}5$.
3. Với dữ liệu hai mặt trăng, thử tổ hợp (C = 100, gamma = 10) và (C = 0,1, gamma = 0,1). Kết quả nào thuộc vùng overfit, vùng nào underfit?
4. Huấn luyện SVM (có chuẩn hóa) cho bài toán tín dụng (dữ liệu one-hot ở bài 2.9). So sánh AUC với Logistic Regression và Random Forest.

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Điểm (3, 3) có $x_1 + x_2 = 6$, nằm **bên kia** đường biên cũ (5,5), tức là "lấn" sang phía lớp 1. Nó trở thành support vector mới, đường biên phải dời về phía lớp 1 và **lề hẹp lại** đáng kể. Với C = 1000 (phạt rất nặng), SVM cố tách đúng: đường biên sẽ nằm giữa (3, 3) và (4, 4), tức khoảng $x_1 + x_2 = 7$.

```
Xd2 = np.vstack([Xd, [3, 3]]); yd2 = np.append(yd, 0)
s2 = SVC(kernel="linear", C=1000).fit(Xd2, yd2)
print(s2.support_vectors_, s2.coef_.round(3), s2.intercept_.round(3), round(2 /
np.linalg.norm(s2.coef_[0]), 3))
# [[3. 3.] [4. 4.]] [[1. 1.]] [-7.] 1.414
```

Kết quả khớp dự đoán: chỉ còn 2 support vector (3, 3) và (4, 4); đường biên $x_1 + x_2 - 7 = 0$, tức $\mathbf{x_1 + x_2 = 7}$; lề hẹp từ 3,536 xuống **1,414**. Một điểm duy nhất sát biên đã kéo lệch cả đường biên: đó là lý do dùng lề mềm (C nhỏ hơn) với dữ liệu thật. 2. $\|x - x'\|^2 = (1-0)^2 + (2-0)^2 = 5$. $K = e^{(-0{,}5 \times 5)} = e^{(-2{,}5)} \approx$ **0,082**. 3. Code: `SVC(kernel="rbf", C=..., gamma=...)` như mục 5.4. Cặp (100, 10): C lớn và gamma lớn → nghiêng về **overfit** (train rất cao, test có thể giảm). Cặp (0,1; 0,1): cả hai nhỏ → **underfit** (train và test đều thấp). 4. Code:

```
svm_tin_dung = make_pipeline(StandardScaler(), SVC(probability=True,
random_state=42))
# fit trên dữ liệu tín dụng one-hot (bài 2.9 mục 5.1), rồi tính roc_auc_score trên
test
```

Ghi lại kết quả. Với 800 mẫu và 48 đặc trưng, SVM thường cho AUC cùng khoảng với Logistic Regression; không có mô hình nào thắng tuyệt đối.

---

9. Flashcard

Mặt trước	**Mặt sau**
SVM tìm gì?	Đường biên có lề (khoảng trống hai bên) rộng nhất.
Support vector là gì?	Các điểm sát đường biên nhất; chỉ chúng quyết định đường biên.
Độ rộng lề (SVM tuyến tính)?	2 / ‖w‖
C nhỏ thì sao?	Lề rộng, chấp nhận vi phạm, đường mượt → có thể underfit.
C lớn thì sao?	Lề hẹp, cố tách đúng mọi điểm → có thể overfit.
Kernel trick là gì?	Tính tích vô hướng trong không gian nhiều chiều hơn mà không cần tạo ra không gian đó.
Kernel phổ biến nhất?	RBF.
gamma lớn thì sao?	Mỗi điểm ảnh hưởng rất hẹp → đường biên lởm chởm, overfit.
SVM có cần chuẩn hóa?	Bắt buộc.
SVM yếu ở đâu?	Dữ liệu rất nhiều dòng (chậm); khó giải thích.

Bài tiếp theo: 2.13 – K-Means.